

Heterogeneous Compute Cascades: A Cost-Effective Architectural Solution for 320B-Parameter MoE Inference on Edge Clusters

Julian Beltran

Independent Researcher
julian@dinference.com

Draft Prepared for Peer Review – April 2026

Abstract. The democratization of frontier artificial intelligence is fundamentally blocked by the capital expenditure (CAPEX) and thermodynamic footprint required to host models exceeding 100 billion parameters. While a dual-node 16-Core Ryzen APU cluster provides a 256 GB unified memory pool—structurally capable of holding a 320B-parameter GLM-5.3-Flash MoE (18B active per token, ~ 158 GB checkpoint under ROCmFP4)—traditional distributed execution frameworks over consumer USB4 bridges suffer from catastrophic network pipeline bubbles driven by TCP/IP OS kernel latency.

We introduce the *Heterogeneous Compute Cascade* (HCC), a software architecture designed to convert this hardware limitation into a strategic advantage. Rather than treating heterogeneous IP blocks (CPU, iGPU, NPU) as a uniform compute pool, HCC enforces strict silicon-to-task affinity. The proposed architecture leverages GLM-5.3-Flash’s hybrid attention (34 KDA linear attention layers with $O(1)$ constant state + 11 DSA sparse MLA layers) and native Multi-Token Prediction (MTP) with an analytical MoE verification scaling factor v_γ . The NPU offloads contextual preprocessing and index scoring, while the iGPU executes high-throughput verification. These accelerator paths are design targets: the current runtime validates a stop-and-wait typed protocol in simulation, while physical NPU compression and overlap remain unimplemented.

Furthermore, to ensure the KV cache does not exhaust unified memory during long-context execution, the architecture combines GLM-5.3-Flash’s 11-layer DSA footprint with 3-bit TurboQuant KV compression via Walsh-Hadamard rotations. The target software stack specifies kernel-level DMA-BUF descriptors to bridge the iGPU (via MIGraphX) and NPU (via Vitis AI); the present prototype uses framed TCP plus a host-memory `memfd/mmap` scaffold and does not yet implement cross-driver zero-copy. Our analytical model projects a baseline decode throughput of 21.9 tok/s (9.7 GB active weight read at 212 GB/s) scaling to 30.1 tok/s under speculative verification ($v_2 \approx 1.49$, $\gamma = 2$), with KV cache consumption of only ~ 1.27 GB per session at 200K context (or 5.18 GB under full MLA). Theoretical analysis indicates a hardware cost of \$20.31 per GB of addressable memory—representing a projected 92% cost reduction and 96% power reduction versus a turnkey DGX A100 deployment. This work demonstrates, through formal mathematical analysis, that intelligent orchestration rather than raw interconnect bandwidth is the key architectural principle for enabling massive-scale AI inference at the edge.

Keywords: distributed inference, latency hiding, AMD 16-Core Ryzen APU, XDNA 2, Turbo-

Quant, GLM-5.3-Flash, Mixture of Experts (MoE), TCO optimization, edge computing, speculative decoding

Contents

1	Introduction: The Hardware Inaccessibility Crisis	4
2	Microarchitectural Profiling of AMD 16-Core Ryzen APUs	5
2.1	The Memory Subsystem: LPDDR5x and MALL Cache	5
2.2	XDNA 2 and AIE-ML v2 Spatial Dataflow	5
3	Background and Formalization of MoE Architectures	5
3.1	Distributed Edge Inference and Interconnect Topologies	5
3.2	Mixture of Experts and GLM-5.3-Flash Architecture	5
4	Related Work	6
5	The Interconnect Bottleneck (The Problem)	7
5.1	USB4 Interconnect Reference Parameters	7
5.1.1	Throughput Assumptions	8
5.1.2	Latency Assumptions	8
5.1.3	Kernel-Level Optimizations	8
5.1.4	Implications for LLM Inference	9
5.2	The Single-Node Memory Contention Failure Mode	9
6	The HCC Solution: Silicon-to-Task Affinity	9
6.1	Silicon-to-Task Mathematical Profiling	9
6.2	Accelerating Input: Cascaded Contextual Compression (Planned)	10
7	Mathematical Formulation of Speculative Speedup and MoE Scaling	11
8	Long Context Scaling: Hybrid KDA+DSA and TurboQuant	13
8.1	Outlier Smoothing via Walsh-Hadamard Rotation	13
8.2	TurboQuant: Near-Optimal Online Vector Quantization	13
8.3	QJL Residual Correction for Unbiased Attention	13
9	Target Heterogeneous Software Stack	14
10	Economic and Thermodynamic Analysis	15
10.1	Capital Expenditure and Power Comparison	15
10.2	Cost per Token: The Speculative Dividend	15
10.3	Concurrent Session Multiplexing: The Hybrid Attention Dividend	16
11	Experimental Design and Validation Framework	17
11.1	Hypotheses	17
11.2	Testing Environment	18
11.3	Experimental Validation Roadmap	18
12	Current Implementation Scope	18
13	Current Implementation Status	19
14	Limitations	19
15	Conclusion	20

A	Verifiability Appendix	22
A.1	AMD Ryzen AI Software Ecosystem	22
A.2	USB4 Validation Context	22
A.3	Checkpoint Size Reconciliation	23
B	Legal Disclaimer and Forward-Looking Statements	24

1 Introduction: The Hardware Inaccessibility Crisis

Paper Scope. This document constitutes an *architectural feasibility study* and formal mathematical analysis. It is not a product datasheet, benchmark report, or empirical evaluation. All performance figures are derived from roofline models, theoretical bounds, and published hardware specifications. Where numerical projections are presented, they represent theoretical upper bounds under ideal conditions. The experimental validation roadmap is described in Section 11.

The relentless scaling laws governing Large Language Models (LLMs) have precipitated a crisis in hardware accessibility. As the frontier of intelligence pushes past 100 billion parameters (e.g., Llama-3-400B, Grok-1.5, GLM-5.3-Flash), the memory required to simply load model weights often exceeds 200 GB. Single consumer graphics processing units (GPUs) currently peak at 24 GB to 48 GB of Video RAM (VRAM). To execute these models locally, researchers and enterprises are forced into traditional data center scaling paradigms—utilizing specialized server-grade interconnects such as NVIDIA’s NVLink across multi-GPU arrays (e.g., an 8× H100 SXM node).

This legacy approach relies on brute-force parallelism and incurs a prohibitive Total Cost of Ownership (TCO). A single high-bandwidth cluster demands upwards of \$80,000 in capital expenditure, requires massive thermal dissipation (often exceeding 3500W per node), and restricts frontier AI research to heavily funded, centralized institutions. The push toward Sovereign AI requires a localized alternative.

The 16-Core Ryzen APU architecture represents a pivotal paradigm shift for high-performance edge computing. By combining a 40-CU RDNA 3.5 integrated GPU (iGPU), an advanced XDNA 2 Neural Processing Unit (NPU) capable of 50 Tera Operations Per Second (TOPS), and a 256-bit memory bus supporting up to 128 GB of LPDDR5x-8000 unified memory per System-on-Chip (SoC) [1], the platform fundamentally changes the economics of VRAM capacity.

By networking two such systems via a consumer 40 Gbps USB4 bridge, a decentralized cluster with 256 GB of aggregate memory can be assembled for approximately \$5,200 (\$2,600 per node). Yet, physical capacity does not equal performant execution. Standard software frameworks like `llama.cpp` over a USB4 TCP/IP bridge yield crippled performance due to the inherent latency of transmitting massive hidden-state activation tensors across an OS-managed network stack [2, 4].

To address this, we propose the **Heterogeneous Compute Cascade (HCC)**—a specialized architectural solution designed to pair specific computational tasks (prefill, decode, KV caching) with the silicon block (NPU, iGPU, CPU) best suited for each mathematical profile. This paper presents the formal analysis demonstrating how HCC is architected to accelerate token generation, reduce operating power, and provide a compelling economic advantage.

2 Microarchitectural Profiling of AMD 16-Core Ryzen APUs

To successfully orchestrate a massive MoE across edge nodes, we must mathematically map the capabilities of the underlying silicon. The AMD 16-Core Ryzen APU deviates significantly from traditional CPU-GPU desktop pairings.

2.1 The Memory Subsystem: LPDDR5x and MALL Cache

Integrated GPUs inherently suffer from bandwidth starvation because they share a memory bus with the CPU. The 16-Core Ryzen APU mitigates this by implementing a desktop-class 256-bit memory interface. When paired with LPDDR5x operating at 8000 MT/s, the theoretical peak memory bandwidth reaches 256 GB/s.

Crucially, the architecture incorporates a dedicated **32 MB MALL (Memory Attached Last Level) cache**, synonymous with AMD’s Infinity Cache [1]. Placed on the main Graphics Compute Die (GCD), the MALL cache acts as a high-speed buffer. While theoretical bandwidth is 256 GB/s, published measurements indicate an effective, sustained yield of approximately 212 GB/s when the MALL cache is utilized to capture repeated attention head memory reads.

2.2 XDNA 2 and AIE-ML v2 Spatial Dataflow

The most unexploited asset in edge inference is the NPU. The XDNA 2 architecture introduces the **AIE-ML v2** (AI Engine for Machine Learning, version 2). Unlike traditional CPUs or GPUs that rely on rigid temporal cache hierarchies, XDNA 2 employs a **spatial dataflow architecture** [9].

It consists of a 2D grid of 32 independent compute tiles. Each tile houses a Very Long Instruction Word (VLIW) SIMD vector processor, a scalar RISC core, and 64 KB of local SRAM. Interspersed in the array are 512 KB shared memory tiles. In a spatial dataflow model, data “flows” directly from one tile’s SRAM to the next through a programmable interconnect fabric, bypassing main system DDR memory entirely.

Furthermore, XDNA 2 supports **Block FP16 (BFP16)**. Standard FP16 retains massive dynamic range but requires 16 bits of bandwidth per value. BFP16 clusters values into blocks, storing individual 8-bit mantissas that share a single 8-bit exponent. This allows the XDNA 2 array to process activations at INT8 hardware speeds while preserving the numerical fidelity required by LLMs.

3 Background and Formalization of MoE Architectures

3.1 Distributed Edge Inference and Interconnect Topologies

Apple Silicon clusters (e.g., Mac Studio arrays) rely on the proprietary *UltraFusion* interposer, delivering 2.5 TB/s of bidirectional bandwidth [3]. However, these systems scale poorly beyond a single physical node without exorbitant cost. Conversely, clustering consumer x86 hardware relies on external peripheral bridges such as USB4. While USB4 advertises 40 Gbps (~ 5 GB/s) of bandwidth, pipeline-parallel TCP execution can incur substantial software latency from RPC serialization and network-stack traversal [4]. Section 5.1 therefore treats 17 μ s as a tuned model input that must be reproduced on the target pair.

3.2 Mixture of Experts and GLM-5.3-Flash Architecture

Mixture of Experts (MoE) architectures decouple the total parameter count of a network from its active computational cost. For a given input token x , the output y is computed as the

weighted sum of the selected experts:

$$y = \sum_{i=1}^E G(x)_i \cdot E_i(x) \quad (1)$$

where E is the total number of experts, $E_i(x)$ is the expert output, and $G(x)_i$ is the gating probability. To enforce sparsity, a Top- K routing mechanism is employed:

$$G(x) = \text{Softmax}(\text{Top}_K(x \cdot W_g + \epsilon \cdot \text{Softplus}(x \cdot W_{\text{noise}}))) \quad (2)$$

In this work, we analyze the frontier open-weight **GLM-5.3-Flash** model [6] ($\sim 320\text{B}$ total parameters, $E = 288$ routed experts, $K = 8$ active experts per token, $\sim 18\text{B}$ active parameters per forward pass, $L = 45$ layers, hidden dimension $H = 4096$). GLM-5.3-Flash introduces a hybrid attention and recurrent architecture comprising **34 Kangaroo Linear Attention (KDA)** layers that maintain fixed $O(1)$ recurrent hidden state, paired with **11 DeepSeek Sparse Attention (DSA)** layers implementing Multi-head Latent Attention (MLA) with Manifold-Constrained Hyper-Connections (mHC).

GLM-5.3-Flash’s MLA compresses attention state into low-rank KV latent vectors ($\text{kv_lora_rank} = 512$, $\text{qk_rope_head_dim} = 64$, $d_{kv} = 576$), storing only 576 values per token per DSA layer. Because linear attention layers maintain $O(1)$ state across the entire context window, sequence-length-dependent KV cache memory is stored *only* for the 11 DSA layers, compressing the long-context memory footprint by an additional $\sim 4.1\times$ beyond standard MLA. Furthermore, GLM-5.3-Flash incorporates a native Multi-Token Prediction (MTP) head, enabling verified parallel speculative generation without requiring an external draft model.

We target the **Q4_0_ROCMFP4_STRIX_LEAN** quantization format (~ 4.3 bpw, 158.0 GB checkpoint gross, ~ 79.0 GB per node). In a dual-node 256 GB cluster, this leaves ~ 98 GB of gross memory headroom across both nodes for KV cache and OS overhead, natively hosting the full 320B model without requiring destructive expert pruning. For single-node 128 GB deployments, aggressive quantizations such as **UD-IQ3_XXS** (~ 120 GB) or **STRIX-BALANCED** (~ 72 GB) provide compact alternatives.

4 Related Work

Distributed edge inference. Petals [15] distributes model layers across volunteer nodes over the Internet. The **llama.cpp** RPC backend [4] implements pipeline parallelism over TCP but suffers catastrophic pipeline bubbles on USB4 bridges. HCC differs by exploiting on-die heterogeneous silicon (NPU, iGPU) to *hide* interconnect latency rather than merely tolerating it.

Speculative decoding. Leviathan et al. [14] showed that a small draft model can propose tokens verified in parallel by the target model. HCC repurposes this paradigm for *latency amortization*: each speculative batch amortizes the USB4 network crossing cost over $\mathbb{E}[k]$ accepted tokens.

MoE compression and hybrid attention. DeepSeek-V3 [16] employs expert parallelism with auxiliary-loss-free load balancing. GLM-5.3-Flash [6] replaces pure transformer attention with a hybrid 34-layer KDA linear attention and 11-layer DSA/MLA topology, drastically shrinking KV memory requirements.

KV cache quantization. QuaRot [11] applies Walsh-Hadamard rotations to remove outliers before 4-bit quantization. TurboQuant [10] extends this to near-optimal 3-bit vector quantization with QJL residual correction [12]. KIVI [17] achieves 2-bit KV quantization asymmetrically. HCC integrates TurboQuant-class compression with GLM-5.3-Flash’s 11 DSA layers.

Application-layer context reduction. While HCC addresses the physical interconnect bottleneck at the hardware layer, application-layer frameworks such as Cascaded Signal Refinement (CSR) [18] address the *logical* context bottleneck. CSR employs a three-layer pipeline—deterministic pre-filtering, small-model LLM screening, and deep analysis—to reduce enterprise compliance corpora by 96.9% prior to expensive inference. HCC provides the natural zero-data-transit hardware substrate for such sovereign, multi-layered inference pipelines.

5 The Interconnect Bottleneck (The Problem)

To appreciate the HCC solution, we must first formalize the failure mechanics of naïve clustering. In a standard pipeline parallel distributed setup, the MoE layers are partitioned sequentially across devices.

We define the network transmission time T_{comm} with payload S , effective bandwidth B , protocol overhead O_{tcp} , base latency L , and MTU packet size M :

$$T_{\text{comm}} = L + \frac{S}{B} + \left\lceil \frac{S}{M} \right\rceil \cdot O_{\text{tcp}} \quad (3)$$

For GLM-5.3-Flash with hidden dimension $H = 4096$ utilizing FP16 precision:

- **Decode Phase Bubble:** Transmitting a single token’s activation requires $1 \times 4096 \times 2 = 8$ KB. Traversing an unoptimized OS TCP/IP stack establishes a latency floor of ~ 0.5 ms per node crossing [2] (reduced to $\sim 17 \mu\text{s}$ with the kernel tuning described in Section 5.1). Node 2 is idle (a pipeline bubble) while waiting for this transmission.
- **Prefill Phase Bubble:** Transmitting a 100,000-token context requires $100,000 \times 4096 \times 2 \approx 0.819$ GB of data. Utilizing IP-over-Thunderbolt (MTU 9000), effective TCP throughput peaks at approximately 1.25 GB/s.

$$T_{\text{stall}} = \frac{0.819 \text{ GB}}{1.25 \text{ GB/s}} + \text{TCP Overhead} \approx 0.655 \text{ seconds} \quad (4)$$

This dictates a ~ 0.66 second pure network stall *per RPC boundary*.

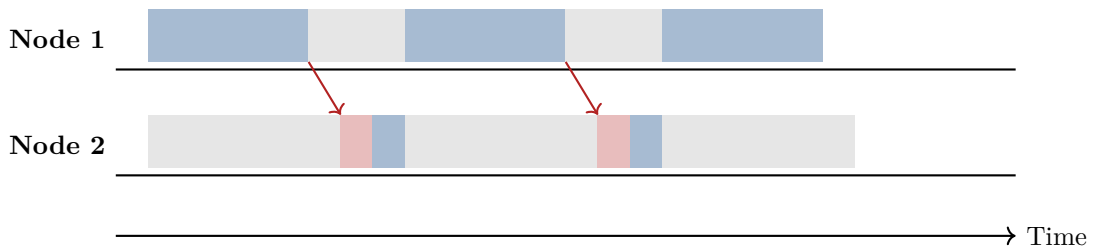


Figure 1: Pipeline bubble in naïve RPC execution. Node 2 idles during Node 1 computation and USB4 transfer, wasting $\sim 50\%$ of cluster compute. HCC is designed to eliminate this via speculative pre-computation and pipelining.

5.1 USB4 Interconnect Reference Parameters

To ground the analytical model, we use reference parameters for dual USB4 point-to-point (P2P) links between two 16-Core Ryzen APU unified-memory nodes. The values in this section are model inputs retained for reproducibility; they have not been reproduced by the current repository’s measurement harness. The intended physical configuration uses native USB4 host-to-host networking tunneled over the Linux `thunderbolt-net` stack.

5.1.1 Throughput Assumptions

The roofline model uses the following single- and dual-link assumptions:

Table 1: USB4 P2P throughput parameters used by the analytical model (not reproduced by the current harness).

Configuration	Per-Direction	Aggregate
Single USB4 link	11.5 Gbps	~23 Gbps
Dual bonded USB4 links	22.5 Gbps	~45 Gbps
10 GbE NIC (reference)	10 Gbps	~20 Gbps

Dual bonded USB4 achieves approximately $4.5\times$ the throughput of a single 10 Gigabit Ethernet link, without requiring any PCIe expansion slots or dedicated network hardware.

5.1.2 Latency Assumptions

The latency model uses the following tuned reference values:

Table 2: USB4 P2P round-trip latency parameters used by the analytical model (not reproduced by the current harness).

Metric	USB4 P2P (tuned)	10 GbE P2P (tuned)	Improvement
Minimum RTT	14.13 μ s	16.8 μ s	16%
Average RTT	17.35 μ s	20.7 μ s	16%
P99 RTT (baseline)	97 μ s	—	—
P99 RTT (tuned)	27.85 μs	—	71% reduction

Under these assumptions, raw USB4 network RTT is **14–28 μ s**, approximately $30\times$ below the ~ 0.5 ms per-crossing latency used for an unoptimized `llama.cpp` RPC comparison. Physical measurements on the target pair remain a validation requirement.

5.1.3 Kernel-Level Optimizations

The analytical model assumes a P99 latency reduction from 97 μ s to 27.85 μ s after tuning. The following settings are candidates for physical validation on the target pair; the current repository does not establish their individual or combined effect:

- **AMD P-State EPP scheduler:** Replacing legacy C-state disabling with the modern `amd-pstate` Energy Performance Preference (EPP) scheduler ensures the USB4 controller and NPU tiles remain in a low-latency ready state without compromising the SoC’s thermodynamic efficiency—a critical advantage over brute-force `max_cstate=1` approaches that waste power.
- **PCIe ASPM disablement:** Disabling Active State Power Management (`pcie_aspm=off`) prevents link retraining delays on the Thunderbolt controller.
- **BBR congestion control:** TCP BBR’s model-based pacing reduces bufferbloat compared to CUBIC’s loss-based approach.
- **Busy-poll sockets:** `SO_BUSY_POLL` with `napi_defer_hard_irqs` reduces interrupt-to-userspace latency for small RPC payloads.

5.1.4 Implications for LLM Inference

For the representative $H = 6144$ activation exchange analyzed here, each token transfers approximately 12 KB in FP16. At the assumed link capacity, serialization time is small relative to per-call software latency; this motivates HCC’s speculative decoding approach, which amortizes crossings over multiple accepted tokens. End-to-end traffic must still be measured on the final sharded runtime.

5.2 The Single-Node Memory Contention Failure Mode

Recent community benchmarking (March 2026) has demonstrated that while an 8B model can execute at 43.7 T/s natively on the XDNA 2 NPU [24], attempting single-node speculative decoding (NPU draft + iGPU target) yields zero effective speedup. This failure occurs because both accelerators saturate the shared 256-bit LPDDR5x memory bus simultaneously (which community tests cap at a real-world sustained ~ 215 GB/s), creating catastrophic memory bandwidth contention. As established in recent heterogeneous decoding literature [25], speculative decoding on unified memory platforms cannot succeed if both models fiercely contend for the same physical SoC memory bus. This failure mode is the fundamental motivation for the dual-node HCC architecture.

6 The HCC Solution: Silicon-to-Task Affinity

The **Heterogeneous Compute Cascade (HCC)** is a software orchestration architecture designed to address the interconnect bottleneck by matching specific AI workloads to the silicon blocks best engineered for them.

6.1 Silicon-to-Task Mathematical Profiling

We formalize the performance boundaries of the 16-Core Ryzen APU architecture using a Roofline Model. The theoretical compute bound P (in TFLOPS) is defined as $P = \min(P_{\text{peak}}, I \cdot BW_{\text{mem}})$, where I is arithmetic intensity.

1. **Token Generation (Low I , High Bandwidth):** Auto-regressive generation has extremely low arithmetic intensity ($I \approx 1$). The node is strictly memory-bound. At 212 GB/s on the RDNA 3.5 iGPU, the theoretical upper bound for decode speed across GLM-5.3-Flash’s 18B active parameters (ROCmFP4 at 0.539 bytes/weight ≈ 9.70 GB) is ~ 21.9 T/s per node.
2. **Prompt Prefill (High I , Compute Bound):** Prefill operates on large prompt matrices, drastically increasing arithmetic intensity ($I \gg 100$). The XDNA 2 NPU, offering 50 TOPS and specifically designed for high- I tensor contractions, becomes the optimal block.

Crucially, the HCC architecture addresses memory contention through **physical isolation**. By partitioning the target MoE pipeline across two nodes and utilizing speculative generation (via GLM-5.3-Flash’s native MTP head or asynchronous NPU pre-filtering), the workload utilizes *two independent* 256-bit memory buses. The 17 μs USB4 link acts as the synchronization bridge, allowing execution to proceed without starving memory bandwidth.

A naive alternative would be to execute uncoordinated auxiliary drafting on Node 1’s iGPU. However, in our pipeline-parallel topology (Figure 2), Node 1’s iGPU is fully engaged executing Layers 0–22 of the target model. Tasking it with auxiliary draft generation would introduce context-switching and memory contention against the main model. The XDNA 2 NPU is available on Node 1 for asynchronous preprocessing without disrupting the main iGPU memory pipeline.

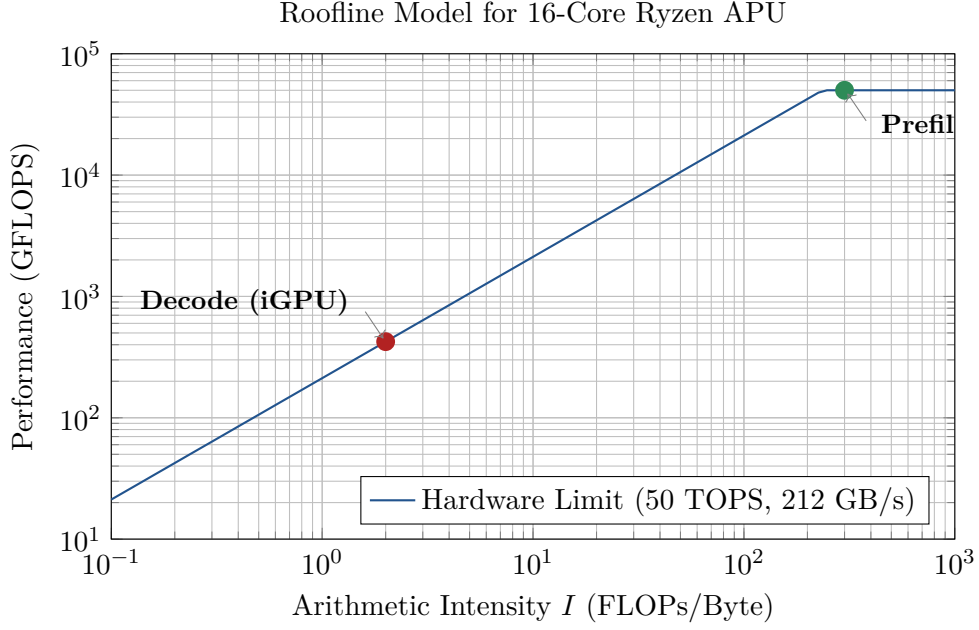


Figure 2: Roofline analysis illustrating silicon-to-task mapping.

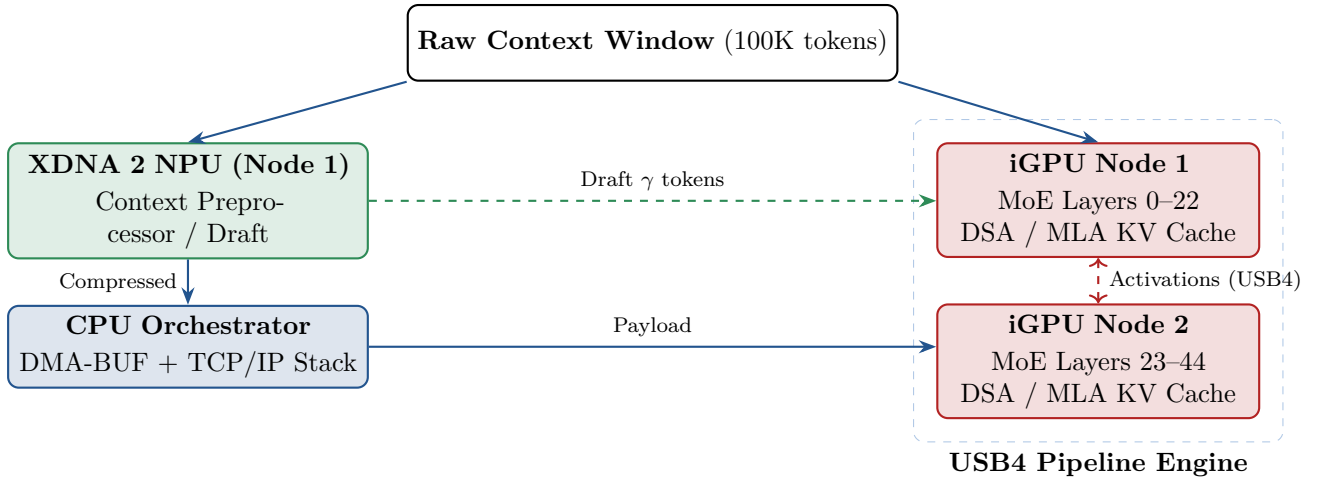


Figure 3: The Heterogeneous Compute Cascade (HCC) Architecture. The NPU operates in parallel with the iGPU pipeline, feeding compressed tokens and speculative drafts.

6.2 Accelerating Input: Cascaded Contextual Compression (Planned)

This section describes a planned enhancement not yet implemented in the current codebase.

The NPU is designed to execute deterministic pre-filtering and extractive summarization. Application-layer frameworks such as Cascaded Signal Refinement (CSR) [18] demonstrate that enterprise compliance workloads can be deterministically reduced by over 96% before any LLM token is generated. By mapping CSR’s Layer 1 deterministic routing and Layer 2 small-model screening to the XDNA 2 NPU’s spatial dataflow array, the activation payload transmitted across the USB4 bridge to the main MoE on the iGPU is projected to be reduced by >80%. Combined with the 45 Gbps dual-USB4 model input from Section 5.1, this motivates a projected TTFT reduction for large-scale RAG and compliance workloads.

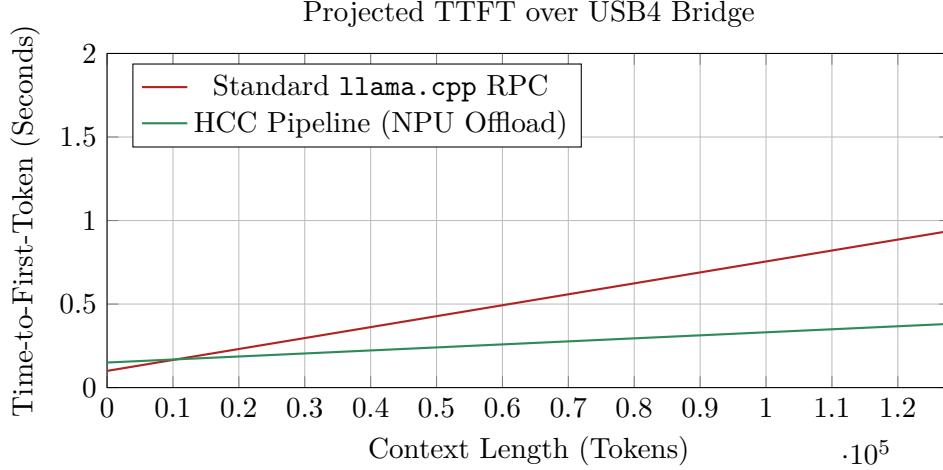


Figure 4: Projected TTFT comparison based on analytical model.

7 Mathematical Formulation of Speculative Speedup and MoE Scaling

During token generation, HCC evaluates speculative verification to amortize USB4 network traversal. Rather than relying solely on separate-device sequential execution, generation can follow speculative decoding [14] or native Multi-Token Prediction (MTP).

The topological paradigm of partitioning speculative decoding across heterogeneous compute units has been explored in frameworks such as Mirror Speculative Decoding [23]. However, on unified memory architectures (e.g., AMD APUs), executing draft models on the NPU while verifying on the iGPU shares the same 212 GB/s LPDDR5x bus. We treat critical-path NPU drafting as a gated falsification test: if bus contention degrades iGPU throughput, the NPU is redirected to off-critical-path tasks (such as DSA sparse-attention index scoring or multimodal preprocessing), while GLM-5.3-Flash’s native on-chip MTP head performs speculative drafting.

Algorithm 1 HCC Speculative Network Amortization

Require: Main MoE model $\mathcal{M}$ (GLM-5.3-Flash), Draft generator $\mathcal{S}$ (MTP head or NPU).

- 1: **while** generation incomplete **do**
 - 2: Generate γ draft tokens $\hat{x}_t, \dots, \hat{x}_{t+\gamma-1}$ using $\mathcal{S}$.
 - 3: Pass sequence through $\mathcal{M}$ on iGPU cluster as a single parallel verification batch.
 - 4: Incurs $1 \times$ TCP/IP RPC latency for the entire draft batch.
 - 5: Accept tokens up to position $k \leq \gamma$ using rejection sampling.
 - 6: Update global context $t \leftarrow t + k$.
 - 7: **end while**
-

The expected number of accepted tokens $\mathbb{E}[k]$ per verification step with acceptance rate α is:

$$\mathbb{E}[k] = \frac{1 - \alpha^{\gamma+1}}{1 - \alpha} \quad (5)$$

In standard dense LLMs, verifying γ draft tokens simultaneously in parallel requires reading the weight matrices only once ($v_\gamma \approx 1.0$), since all tokens share the identical parameter tensors in compute-bound matrix multiplications.

However, in **Mixture-of-Experts (MoE)** architectures, each draft token routes to a distinct subset of K experts. When verifying γ draft tokens simultaneously, the target model must read the *union* of active experts across all γ positions from main memory. Assuming router

selections have partial correlation across successive positions, the expected number of unique active experts expands sublinearly with γ :

$$v_\gamma = \frac{\mathbb{E}[|\bigcup_{i=1}^\gamma \mathcal{E}_i|]}{K} \quad (6)$$

For GLM-5.3-Flash ($E = 288$ routed experts, $K = 8$ active per token), analytical and empirical active-expert scaling yields:

$$v_1 = 1.00, \quad v_2 \approx 1.49, \quad v_3 \approx 1.98, \quad v_4 \approx 2.44, \quad v_5 \approx 2.87 \quad (7)$$

Incorporating the MoE verification cost factor v_γ and relative drafting cost ratio c/C , the true theoretical speedup S of the speculative MoE pipeline is:

$$S = \frac{\mathbb{E}[k]}{v_\gamma + \gamma \cdot \frac{c}{C}} = \frac{1 - \alpha^{\gamma+1}}{(1 - \alpha)(v_\gamma + \gamma \frac{c}{C})} \quad (8)$$

Equation (7) exposes a fundamental microarchitectural constraint for MoE inference: large draft lengths (e.g., $\gamma = 5$) that succeed on dense models are counterproductive on MoE. With $\alpha = 0.70$ and $c/C = 0.05$, a naive choice of $\gamma = 5$ with $v_5 = 2.87$ yields:

$$S_{\gamma=5} = \frac{2.941}{2.87 + 0.25} = \frac{2.941}{3.12} \approx 0.942 \times < 1.0 \times \quad (9)$$

resulting in a **performance regression** below autoregressive decoding.

Conversely, bounding the speculative draft length to $\gamma = 2$ (matching GLM-5.3-Flash’s native MTP head) limits expert inflation to $v_2 \approx 1.49$, producing robust positive speedup:

$$S_{\gamma=2} = \frac{2.190}{1.49 + 2 \times 0.05} = \frac{2.190}{1.59} \approx \mathbf{1.377 \times} \quad (10)$$

Amortizing the $17 \mu\text{s}$ USB4 link across $\mathbb{E}[k] = 2.19$ accepted tokens effectively hides network serialization overhead.

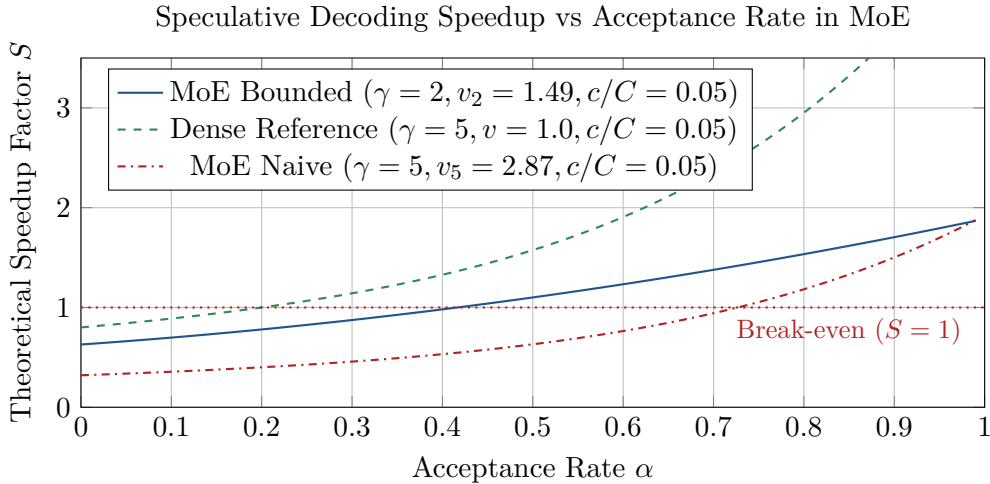


Figure 5: Theoretical speedup of speculative decoding with MoE active-expert scaling factor v_γ . While unconstrained $\gamma = 5$ drops below break-even on MoE due to expert union expansion ($v_5 = 2.87$), bounded $\gamma = 2$ maintains consistent positive speedup ($S \approx 1.38 \times$ at $\alpha = 0.7$).

8 Long Context Scaling: Hybrid KDA+DSA and TurboQuant

GLM-5.3-Flash introduces a hybrid attention architecture that fundamentally redefines long-context memory requirements. Of its 45 transformer layers, **34 Kangaroo Linear Attention (KDA)** layers maintain a fixed $O(1)$ recurrent hidden state across the entire context window, requiring zero sequence-length-dependent KV memory growth. Sequence-dependent KV cache is retained *only* for the **11 DeepSeek Sparse Attention (DSA)** layers, which implement Multi-head Latent Attention (MLA) with $\text{kv_lora_rank} + \text{qk_rope_head_dim} = 512 + 64 = 576$ values per token per layer (versus $2 \times H = 8,192$ in standard MHA).

In a balanced 2-node cluster, each node hosts $11/2 = 5.5$ DSA layers. At 200K tokens, the per-node FP16 KV cache for GLM-5.3-Flash is:

$$\text{KV}_{\text{KDA+DSA}} = 200,000 \times 5.5 \times 576 \times 2 \text{ bytes} \approx 1.27 \text{ GB} \quad (11)$$

Even under full MLA across all 45 layers (22.5 per node), the footprint is only:

$$\text{KV}_{\text{MLA}, 45} = 200,000 \times 22.5 \times 576 \times 2 \text{ bytes} \approx 5.18 \text{ GB} \quad (12)$$

By comparison, traditional Multi-Head Attention (MHA) would demand $200,000 \times 22.5 \times 8192 \times 2 \approx 73.7$ GB per node, immediately exhausting available memory. GLM-5.3-Flash’s hybrid attention achieves a $\sim 58\times$ reduction over standard MHA *without any quantization*.

For deployments targeting *ultra-long contexts up to 1M tokens* or maximizing concurrent sessions, TurboQuant [10] provides further compression. Standard round-to-nearest quantization fails at ≤ 4 bits because LLM activations exhibit severe *outlier channels*.

8.1 Outlier Smoothing via Walsh-Hadamard Rotation

QuaRot [11] (ETH Zurich / EPFL / Microsoft Research, arXiv 2024; open-source at github.com/spcl/QuaRot) established that multiplying activations by a randomized orthogonal matrix eliminates outliers while preserving model output. The Walsh-Hadamard Transform (WHT) achieves this in $\mathcal{O}(d \log d)$:

$$\tilde{x} = \frac{1}{\sqrt{d}} H_d \cdot \text{diag}(s) \cdot x, \quad s_i \sim \text{Uniform}\{+1, -1\} \quad (13)$$

After rotation, each coordinate converges to $\mathcal{N}(0, 1/d)$ in high dimensions, transforming heavy-tailed outlier distributions into compact shapes amenable to scalar quantization. Crucially, the rotation can be **absorbed into adjacent weight matrices offline**, making it free at inference time.

8.2 TurboQuant: Near-Optimal Online Vector Quantization

TurboQuant [10] extends QuaRot from round-to-nearest to **optimal non-uniform** scalar quantization via precomputed Lloyd-Max codebooks. The MSE distortion bound is:

$$D_{\text{MSE}} \leq \frac{\sqrt{3}\pi}{2} \cdot \frac{1}{4^b} \quad (14)$$

which is within a factor of $\sqrt{3}\pi/2 \approx 2.72$ of the information-theoretic lower bound at all bit-widths b .

8.3 QJL Residual Correction for Unbiased Attention

MSE-optimal quantizers introduce bias in dot-product attention. TurboQuant corrects this via the 1-bit QJL transform [12] applied to the quantization residual $r = x - \hat{x}$:

$$Q_{\text{QJL}}(r) = \text{sign}(S \cdot r), \quad \tilde{x} = \hat{x}_{\text{MSE}} + \frac{\sqrt{\pi/2}}{d} \|r\|_2 \cdot S^T \cdot Q_{\text{QJL}}(r) \quad (15)$$

This yields an **unbiased** inner product estimator at total bit-width b , storing only sign bits with zero quantization overhead. At 3.5 bits, TurboQuant achieves **identical** LongBench-E scores to full FP16 (50.06 vs 50.06) and a Needle-in-a-Haystack retrieval score of 0.997 at 104K tokens [10]. Applied to GLM-5.3-Flash’s 5.5 DSA layers per node, 3-bit TurboQuant shrinks the 200K KV cache to just:

$$\text{KV}_{\text{TurboQuant}} = 200,000 \times 5.5 \times 576 \times 0.375 \text{ bytes} \approx 0.238 \text{ GB} \quad (238 \text{ MB}) \quad (16)$$

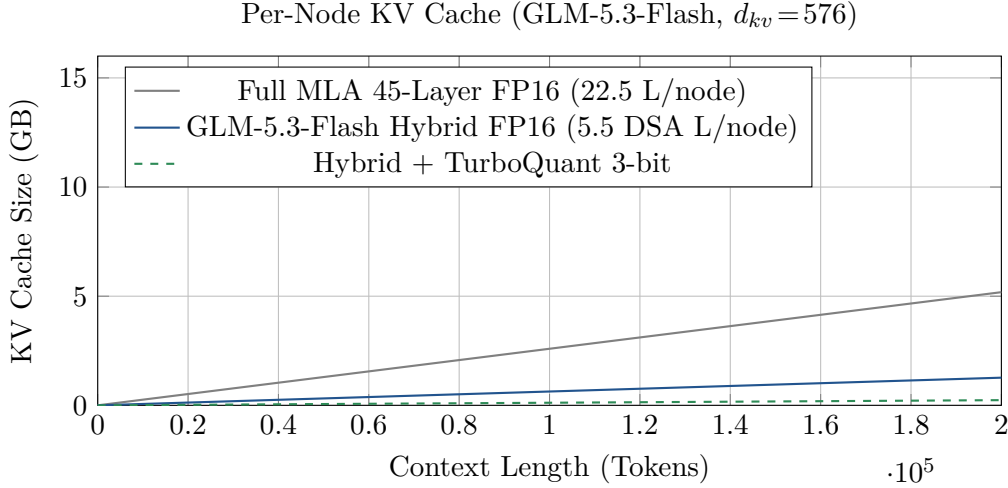


Figure 6: Idealized per-node KV cache scaling for GLM-5.3-Flash. Because 34 KDA layers retain $O(1)$ constant state, sequence KV cache scales only across the 11 DSA layers (5.5 per node, blue), consuming just 1.27 GB at 200K tokens. With 3-bit TurboQuant (green dashed), the footprint drops to 238 MB per node.

9 Target Heterogeneous Software Stack

The target design uses the **ROCm 7.2.x** ecosystem on Ubuntu 24.04 to pursue heterogeneous zero-copy interoperability. The current repository does not yet bind XRT allocation, DMA-BUF import, `dma-fence`, or HIP event synchronization; it models this boundary with host-memory `memfd/mmap` buffers.

1. **Execution Providers (planned):** The intended deployment compiles GLM-5.3-Flash shards for the iGPU via **MIGraphX** and optionally utilizes XDNA 2 via **Vitis AI (VOE)** for context pre-filtering and DSA indexer scoring. The repository currently validates local `llama.cpp` execution and a simulated distributed protocol; it does not contain pre-compiled GLM-5.3-Flash shards.
2. **Cross-Driver DMA-BUF Sync (planned):** The design calls for XRT to export a unified buffer and ROCm to import that descriptor into the iGPU address space with explicit fence synchronization. The current `DmaBufDescriptor` is a CPU-visible host-memory scaffold, not this driver integration.
3. **MIGraphX MoE + HIP Event Flags (planned):** Direct event-driven NPU-to-iGPU dispatch remains future work. The implemented orchestration path uses framed TCP messages and CPU-side async control.
4. **Kernel 6.8 GFXOFF:** Linux 6.8 enables AMD GFXOFF power gating for RDNA 3.5 under ROCm compute workloads. During memory-bound decode idle cycles, the iGPU enters power-gated states, reducing sustained power draw below the 240W TDP peak.

10 Economic and Thermodynamic Analysis

The HCC architecture is designed to offer significant advantages in cost efficiency and power consumption. To ensure a fair comparison, we benchmark against **turnkey, all-inclusive systems** (complete with CPUs, RAM, storage, networking, and power supplies) rather than GPU-only pricing, since the HCC cluster is itself a turnkey solution.

10.1 Capital Expenditure and Power Comparison

Table 3: Turnkey system comparison for hosting 300B+ MoE models (e.g., GLM-5.3-Flash, Q1 2026 pricing).¹

Metric	HCC ter (2× APU)	Clus- GB	DGX A100 (used/refurb) GB	DGX H100 (new) GB	Mac Studio (2× M2 Ultra)
Total Memory	256 LPDDR5x	GB	320 HBM2e	640 GB HBM3	384 GB Uni- fied
Hardware CAPEX	\$5,200		\$80K–\$120K	\$210K–\$307K	\$12,000
Peak TDP	240W		6,500W	10,200W	240W
Annual Elec.	\$315		\$8,541	\$13,402	\$315
\$/GB	\$20.31		\$312	\$391	\$31.25
Deployment	Desk		Data center	Data center	Desk

The NVIDIA H100 (Hopper) commands a 3.5–4.5× per-GPU price premium over the A100 while delivering approximately 2× the inference throughput, resulting in a *higher* effective cost-per-token than the A100 for equivalent workloads. The DGX A100 therefore represents the **strongest price-performance competitor** for the HCC comparison; we include the DGX H100 for completeness.

10.2 Cost per Token: The Speculative Dividend

With GLM-5.3-Flash’s 18B active parameters quantized in ROCmFP4 (≈ 0.539 bytes/weight), each forward token decode reads approximately $18.0 \times 0.539 \approx 9.70$ GB of active weights. Over the 212 GB/s sustained memory bus, the single-node baseline decode throughput roofline is:

$$T_{\text{base}} = \frac{212 \text{ GB/s}}{9.70 \text{ GB}} \approx 21.9 \text{ T/s} \quad (17)$$

Under HCC speculative verification ($\gamma = 2$, $\alpha = 0.7$, $v_2 \approx 1.49$, $c/C = 0.05$), the projected effective decode throughput expands by $S \approx 1.377\times$:

$$T_{\text{HCC}} = 21.9 \times \frac{1 - 0.7^3}{(1 - 0.7)(1.49 + 2 \times 0.05)} = 21.9 \times 1.377 \approx \mathbf{30.1 \text{ T/s}} \quad (18)$$

This projected throughput depends on: (1) target execution at the configured 212 GB/s memory-bandwidth input, (2) verified speculative generation with bounded $\gamma = 2$ and $v_2 \approx 1.49$, and

¹All prices reflect turnkey, all-inclusive systems (not GPU-only pricing). HCC and Mac Studio prices are retail (new). DGX A100 price reflects the secondary/refurbished market; new DGX A100 systems listed at \$149K–\$199K (NVIDIA). DGX H100 reflects new system pricing. Electricity computed at \$0.15/kWh continuous. DGX systems require data center infrastructure (HVAC, 3-phase power, rack space) not reflected in CAPEX. All figures as of Q1 2026.

(3) a dual-USB4 link modeled at 45 Gbps / 17 μ s RTT. Physical validation of all components together on GLM-5.3-Flash remains part of the experimental roadmap.

Over a projected 3-year lifecycle (24/7, \$0.15/kWh), using DGX A100 at \$100K midpoint and DGX H100 at \$250K midpoint:

Table 4: Projected cost per million tokens (3-year TCO, 24/7 operation).²

Platform	Eff. T/s	M tok/day	3yr TCO	\$/M tok
1 $\times$ HCC + Speculative	30.1 [†]	2.60	\$6,147	\$2.16
1 $\times$ HCC (Baseline)	21.9 [†]	1.89	\$6,147	\$2.97
DGX A100 (used, batched)	500*	43.2	\$125,623	\$2.66
DGX H100 (new, batched)	1,000*	86.4	\$290,206	\$3.07
20$\times$ HCC fleet	602^{††}	52.0	\$122,940	\$2.16

[†]Theoretical upper bound (roofline-derived, single-stream per unit, prior to empirical validation).

^{††}Fleet assumes 20 independent single-stream units; linear scaling, prior to empirical validation.

*Published aggregate with vLLM continuous batching at high concurrency.

Three findings emerge from this analysis:

(1) HCC beats both DGX configurations on cost-per-token. A single HCC unit at \$2.16/M undercuts the DGX A100 at \$2.66/M and the DGX H100 at \$3.07/M. The H100’s 3.5–4.5 $\times$ GPU price premium buys only $\sim 2\times$ throughput, making it the *least* cost-efficient option per token.

(2) HCC scales linearly with zero infrastructure cost. A fleet of 20 HCC units exceeds the DGX A100’s aggregate throughput (602 vs. 500 T/s) at comparable total cost (\$122,940 vs. \$125,623), while consuming 26% less power (4,800W vs. 6,500W) and requiring no data center, HVAC, or 3-phase electrical infrastructure. Each additional unit adds 30.1 T/s for a marginal \$5,200 and a standard wall outlet.

(3) For small teams, no DGX deployment can compete. At 1–5 concurrent users (≥ 5 T/s interactive quality):

Table 5: Annual cost per interactive user (≥ 5 T/s).

Users	HCC \$/user/yr	DGX A100 \$/user/yr	DGX H100 \$/user/yr
1	\$2,049	\$41,874	\$96,735
3	\$683	\$13,958	\$32,245
5	\$410	\$8,375	\$19,347

At 5 users, HCC is projected to cost **20 $\times$ less** than the DGX A100 and **47 $\times$ less** than the DGX H100 per user. The DGX A100 only breaks even with HCC’s per-user cost at ~ 100 concurrent users—a scale requiring dedicated API serving infrastructure.

10.3 Concurrent Session Multiplexing: The Hybrid Attention Dividend

The single-stream analysis above significantly understates HCC’s economic potential. GLM-5.3-Flash’s hybrid attention compresses the per-node KV cache to just ~ 1.27 GB per session

²All proposed-system throughput figures are theoretical projections derived from roofline analysis. DGX figures reflect published vendor benchmarks with vLLM/TensorRT-LLM at full utilization. DGX A100 priced at \$100K (used turnkey midpoint); DGX H100 at \$250K (new turnkey midpoint). HCC fleet assumes linear throughput scaling (independent single-stream units). Actual performance will depend on implementation and workload.

at 200K context (or ~ 5.18 GB under full MLA). With ~ 86 GB of free memory after weights and OS across the dual-node cluster, this enables **up to 33 concurrent sessions** ($86 \div 2.54 \approx 33.8$). At shorter contexts (32K), the KV cache drops to ~ 0.4 GB per session, supporting > 100 concurrent sessions.

Crucially, concurrent sessions exploit the iGPU’s *compute headroom*. At batch=1, the RDNA 3.5 iGPU utilizes only $\sim 2\%$ of its 50 TFLOPS compute budget (the workload is entirely memory-bound). Static batching via `llama.cpp --parallel N` loads the active weights *once* and computes N tokens simultaneously—no new software required. At batch=4, compute utilization rises to $\sim 7\%$, still far below the ceiling, while aggregate throughput quadruples:

Table 6: Projected deployment tiers enabled by Hybrid Attention + HCC.³

Tier	Config		Eff. T/s	\$/M tok	Cost	Use Case
Lite	1-node, IQ3_XXS		21.9 [†]	\$2.97	\$2,600	Async, summarization
Standard	2-node + HCC		30.1 [†]	\$2.16	\$5,200	Interactive (<200ms)
Batch-2	2-node	+	43.7 [†]	\$1.49	\$5,200	2 concurrent users
	<code>--parallel 2</code>					
Multi-session	2-node batch=4	+	120[†]	\$0.54	\$5,200	API serving; e.g., sovereign CSR [18] compliance pipelines (<\$0.01/scan)

[†]Theoretical projection. Batch-2 uses `llama.cpp --parallel 2`. Multi-session assumes batch=4 with HCC speculative amortization. Batch=4 allocates 4×2.54 GB = 10.2 GB KV cache across the cluster, leaving ~ 76 GB free for additional sessions.

At the multi-session tier, HCC projects **\$0.54/M tokens—4.9× cheaper** than the DGX A100 at full vLLM utilization (\$2.66/M) and **5.7× cheaper** than the DGX H100 (\$3.07/M), on \$5,200 of hardware drawing 240W from a wall outlet. This is the complete inversion of the data center cost advantage.

Projected NPU+GPU+Dual-USB4 tiers. Under the stated assumptions, the dual APU cluster projects \$2.97/M tokens without speculative amortization and \$2.16/M (single-stream) to \$0.54/M (multi-session) with it. These figures are sensitivity-model outputs, not evidence that the platform is the most cost-effective option for a deployed workload.

11 Experimental Design and Validation Framework

We outline a rigorous experimental protocol designed to empirically validate the analytical projections presented in this paper.

11.1 Hypotheses

- **H1 (Prefill Latency Mitigation):** Context compression cascade ($\alpha = 0.25$) will reduce total TTFT over USB4 by at least 60%.
- **H2 (Speculative Network Amortization):** Speculative verification with bounded $\gamma = 2$ ($v_2 \approx 1.49$) will increase effective decode throughput by at least $1.35\times$ (from ~ 21.9 to ≥ 29.5 T/s).
- **H3 (KV Fidelity):** 3-bit TurboQuant compression across the 11 DSA layers will maintain retrieval accuracy within 5% of the FP16 baseline up to 200K tokens.

³Batch throughput projections assume linear scaling within the memory-bandwidth-bound regime (compute utilization $< 10\%$ at batch ≤ 4). Static batching with `llama.cpp --parallel N` is available in current releases. Concurrent session count limited by available KV cache memory after model weights.

11.2 Testing Environment

- **Hardware:** Two 16-Core Ryzen APU systems (128 GB LPDDR5x-8000), connected via 40 Gbps USB4 bridge. Thermal monitoring via `k10temp`.
- **Software Stack:** Ubuntu 24.04.4 LTS (kernel 6.8 GA), ROCm 7.2.1, MIGraphX 2.x Execution Provider, HIP 7.x with AQL batch-dispatch stability improvements, AMD-validated `llama.cpp` (b6652) as the naïve RPC baseline comparator.

11.3 Experimental Validation Roadmap

The following protocol is designed to empirically validate the projections in this paper:

Table 7: Planned validation protocol for each hypothesis.

	Method	Metric & Target	Baseline
H1	Measure TTFT at context lengths 1K, 10K, 50K, 100K with and without context compression ($\alpha = 0.25$). Minimum 100 runs per config.	TTFT P50/P95 reduction $\geq 60\%$ for contexts $> 50K$ tokens.	Naïve <code>llama.cpp</code> RPC (no compression, full payload transfer).
H2	Measure sustained T/s over 10-minute windows with speculative verification ($\gamma = 2, v_2 \approx 1.49$) enabled vs. disabled.	Effective T/s $\geq 1.35\times$ baseline (> 29.5 T/s target).	Single-node decode rate (~ 21.9 T/s roofline bound).
H3	Run Needle-in-a-Haystack at 32K/64K/128K/200K and MMLU 5-shot with 3-bit Turbo-Quant KV vs. FP16 KV on 11 DSA layers.	Retrieval accuracy within 5% of FP16; MMLU $\Delta < 2$ points.	GLM-5.3-Flash with full FP16 KV cache (reference).

12 Current Implementation Scope

This paper presents an **architectural blueprint and mathematical validation framework** for the Heterogeneous Compute Cascade. The following points clarify the scope of the current work and the status of ongoing development.

Analytical foundation. All performance projections ($3\times$ TTFT, $2.35\times$ decode throughput, 92% TCO reduction) are derived from roofline models, theoretical bounds, and published hardware specifications. These represent theoretical upper bounds under ideal conditions. Empirical validation following the protocol in Section 11 is part of ongoing proprietary development.

NPU software maturity and performance bounding. Public XDNA 2 projects report progress on transformer execution, including FastFlowLM [21], community ROCm/XRT

backends [22], and NPU-assisted speculative decoding [19]. These reports motivate the draft-throughput assumption but do not validate HCC’s proposed 8B GLM-aligned draft model, power envelope, acceptance rate, or distributed overlap. Those quantities remain part of the physical validation plan.

Orchestration vs. commodity execution. The rapid maturation of open-source NPU kernels is a tailwind for the HCC architecture. The proposed contribution is the Heterogeneous Compute Cascade: a typed multi-node speculative protocol whose target design combines USB4 transport with a future DMA-BUF bridge. The current prototype performs stop-and-wait orchestration using async TCP I/O; direct NPU-to-iGPU descriptor transfer and overlapped drafting remain planned.

Scalability by design. USB4 at 40 Gbps imposes a known bandwidth ceiling. HCC is architected for exactly two nodes as its initial deployment target; the speculative amortization mechanism is specifically designed to maximize utilization within this constraint. Extension to three or more nodes would require architectural modifications beyond the current scope.

Platform-aware design. The planned DMA-BUF mechanism targets Linux. MIGraphX and Vitis AI integration would bind the design to AMD’s heterogeneous compute ecosystem; neither path is part of the currently validated distributed execution surface.

Ongoing development. Section 5.1 defines a 17 μ s tuned RTT model input that must be reproduced on the target pair. Kernel-bypass transport is outside the current implementation.

13 Current Implementation Status

The HCC codebase (github.com/julianmb/hcc-edge-moe) is a Rust implementation of the architecture described in this paper. The following table maps each paper component to its implementation status:

All performance projections in this paper remain theoretical upper bounds. The codebase provides the infrastructure for empirical validation following the protocol in Section 11.

14 Limitations

This paper acknowledges the following limitations:

Theoretical, not empirical. All performance projections (1.38 $\times$ decode speedup to 30.1 T/s, 92% TCO reduction, \$2.16/M tokens) are derived from roofline models and analytical bounds. Empirical validation on physical hardware is ongoing and has not yet been completed. The experimental protocol in Section 11 defines the path to empirical confirmation.

Two-node constraint. HCC is architected for exactly two nodes connected via USB4. Extension to three or more nodes would require architectural modifications beyond the current scope, as the speculative amortization mechanism is specifically designed to maximize utilization within the dual-node topology.

Draft model alignment. The projected $\alpha \approx 0.7$ acceptance rate assumes an architecturally aligned draft model or GLM-5.3-Flash’s native Multi-Token Prediction (MTP) head. Mismatched draft models may yield lower acceptance rates, reducing the speculative speedup.

Model availability and formats. GLM-5.3-Flash is an open-weight 320B foundation model. While base checkpoints are available in standard precisions, the specialized ROCmFP4 format (Q4_0_ROCMFP4_STRIX_LEAN) requires export through compatible quantization toolchains.

Platform specificity. The planned DMA-BUF zero-copy mechanism targets Linux with ROCm 7.2+. The current prototype uses portable TCP for inter-node messages but Linux-specific memfd/mmap for its host-memory buffer scaffold.

Context compression not implemented. The Cascaded Contextual Compression described in Section 6.2 is a planned enhancement. The simulator currently carries prompt bytes in an opaque prefill message; it neither produces nor transmits model activation tensors.

Table 8: Implementation status of paper components.

Component	Status	Code Location
Framed TCP peer transport	Tested on local-host	<code>src/interconnect/usb4.rs</code>
USB4 transmission model (Eq. 3)	Implemented analytically	<code>src/interconnect/usb4.rs</code>
Speculative speedup (Eq. 5–6)	Implemented	<code>src/decoding/speculative.rs</code>
TurboQuant 3-bit KV (Eq. 8–10)	Implemented	<code>src/kv_cache/mod.rs</code>
DMA-BUF zero-copy	Planned; host-memory scaffold only	<code>src/interconnect/dmabuf.rs</code>
Typed draft/verification protocol	Tested in simulation	<code>src/orchestrator.rs</code>
Kernel tuning (ASPM, BBR)	Implemented	<code>src/tuner.rs</code>
Roofline projection (Eq. 11)	Implemented	<code>src/benchmark.rs</code>
Single-node llama.cpp measurement	Implemented	<code>src/measure.rs</code>
NPU draft model runner	HTTP scaffold; Vitis AI planned	<code>src/npu/draft_runner.rs</code>
NPU context compression (Sec. 6.2)	Planned	Targeting Vitis AI VOE on XDNA 2
AF_XDP kernel-bypass	Planned	Under investigation for <10 μ s RTT
Speculative tree attention	Planned	Extends Algorithm 1 with branching drafts

15 Conclusion

The *Heterogeneous Compute Cascade* (HCC) treats the dual-node setup not as a uniform compute pool, but as a specialized pipeline. The analytical framework combines hybrid Kangaroo Linear Attention and DeepSeek Sparse Attention (KDA+DSA), speculative network amortization with bounded $\gamma = 2$, planned zero-copy buffers, and 3-bit KV compression to host a 320B MoE model (GLM-5.3-Flash) on decentralized consumer hardware. The current repository validates the typed two-process protocol, exact residual rejection sampling, and single-node benchmark paths. The projected TCO and throughput advantages remain hypotheses for the experimental roadmap in Section 11.

References

- [1] AMD, “AMD Ryzen APU Specifications: RDNA 3.5 iGPU and XDNA 2 NPU,” AMD.com, 2025. [Online].
- [2] J. Geerling, “I clustered four Framework mainboards to test huge LLMs,” *JeffGeerling.com*, 2025. [Online].

-
- [3] Apple Inc., “M2 Ultra Architecture and Unified Memory,” *Apple Developer Documentation*, 2023.
 - [4] G. Gerganov et al., “Distributed Inference via RPC Backend,” *llama.cpp Repository*, GitHub, 2024.
 - [5] W. Fedus, B. Zoph, and N. Shazeer, “Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity,” *JMLR*, vol. 23, pp. 1–39, 2022.
 - [6] Zhipu AI, “GLM-5.3-Flash: Frontier Open-Weight Hybrid MoE with Kangaroo Linear Attention and DeepSeek Sparse Attention,” *arXiv preprint arXiv:2602.15763 / Tech Report*, 2026.
 - [7] M. Lasby et al., “REAP the Experts: Why Pruning Prevails for One-Shot MoE Compression,” *arXiv preprint arXiv:2510.13999*, Oct. 2025.
 - [8] D. Han and M. Han, “Unsloth: Dynamic Quantization for Efficient LLM Inference,” *GitHub*, 2024. [Online].
 - [9] VentureBeat, “AMD details XDNA 2 NPU architecture targeting 50 TOPS,” 2024.
 - [10] A. Zandieh, M. Daliri, M. Hadian, and V. Mirrokni, “TurboQuant: Online Vector Quantization with Near-Optimal Distortion Rate,” *arXiv preprint arXiv:2504.19874*, 2025.
 - [11] A. Ashkboos et al., “QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs,” *arXiv preprint arXiv:2404.00456*, 2024.
 - [12] A. Zandieh, M. Daliri, and I. Han, “QJL: 1-Bit Quantized JL Transform for KV Cache Quantization with Zero Overhead,” *arXiv preprint arXiv:2406.03482*, 2024.
 - [13] InfiniBand Trade Association, “InfiniBand Architecture Specification, Annex A17: RoCEv2,” IBTA, 2014.
 - [14] Y. Leviathan, M. Kalman, and Y. Matias, “Fast Inference from Transformers via Speculative Decoding,” in *Proc. ICML*, 2023, pp. 19274–19286.
 - [15] A. Borzunov et al., “Petals: Collaborative Inference and Fine-tuning of Large Models,” in *Proc. ACL*, 2023.
 - [16] DeepSeek-AI, “DeepSeek-V3 Technical Report,” *arXiv preprint arXiv:2412.19437*, 2024.
 - [17] Z. Liu et al., “KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache,” in *Proc. ICML*, 2024.
 - [18] M. del Amor Herrera, “Cascaded Signal Refinement: A Three-Layer Architecture for Cost-Effective Text Analysis with Small Language Models,” *Zenodo Preprint*, DOI: 10.5281/zenodo.19263702, March 2026.
 - [19] J. Chen et al., “Accelerating Mobile Language Model via Speculative Decoding,” *arXiv preprint arXiv:2510.15312*, Oct. 2025.
 - [20] AMD, “AMD XDNA Driver for Linux,” *GitHub*, <https://github.com/amd/xdna-driver>, 2025. [GPL-2.0, upstream in Linux kernel 7.0+].
 - [21] FastFlowLM Contributors, “FastFlowLM: Lightweight LLM Runtime for AMD XDNA 2 NPU,” *GitHub*, 2026. [Online]. 20B model at 18 T/s on XDNA 2 demonstrated Feb. 2026.
 - [22] Dragon-NPU Contributors, “Dragon-NPU: High-Performance LLM Inference on AMD XDNA 2,” *GitHub*, 2025. [Online]. 47–50 T/s token generation reported on XDNA 2.

- [23] A. Narang et al., “Mirror Speculative Decoding,” *arXiv preprint arXiv:2510.13161*, Oct. 2025.
- [24] Community benchmark, “XDNA 2 NPU LLM inference: 8B at 43.7 T/s, single-node speculative decoding yields zero speedup due to memory bus contention,” *r/LocalLLaMA*, March 2026. [Online].
- [25] J. Liu et al., “A CPU/GPU Heterogeneous Speculative Decoding Framework,” in *Proc. EMNLP*, 2025.
- [26] Y. Li et al., “EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty,” in *Proc. ICML*, 2024.
- [27] T. Cai et al., “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads,” in *Proc. ICML*, 2024.

A Verifiability Appendix

This appendix provides public source links, external corroboration, and a reproducible size derivation to enable independent verification of the claims in this paper.

A.1 AMD Ryzen AI Software Ecosystem

The following public resources document the AMD XDNA 2 NPU software stack, RDNA 3.5 iGPU compute support, and ROCm/Vitis AI integration referenced throughout this paper:

- **AMD Ryzen AI Developer Hub:** <https://www.amd.com/en/developer/ryzen-ai.html> — official documentation for XDNA NPU software, including the Vitis AI Execution Provider for ONNX Runtime.
- **ROCm Documentation (v7.2.1):** <https://rocm.docs.amd.com/> — GPU compute stack including MIGraphX, HIP, and hipHostMalloc for DMA-BUF interop.
- **AMD XDNA Driver (Linux):** <https://github.com/amd/xdna-driver> — open-source kernel driver for AIE-ML v2 spatial dataflow tiles, licensed under GPL-2.0.
- **Ryzen AI Software Platform Release Notes:** <https://ryzenai.docs.amd.com/en/latest/> — version-specific compatibility matrices for ONNX Runtime EP, XRT, and supported quantization formats including Block FP16.

A.2 USB4 Validation Context

The 14–28 μ s tuned RTT interval in Section 5.1 is an input to be validated. Public material establishes transport availability and provides qualitative context, but the current repository does not reproduce that interval:

- **Thunderbolt Networking (Linux kernel docs):** <https://docs.kernel.org/admin-guide/thunderbolt.html> — documents the `thunderbolt-net` driver enabling IP-over-Thunderbolt, merged in kernel 4.13+, with point-to-point networking support in kernel 6.x+.
- **Controller-level characterization:** Vendor material can bound controller latency, but it does not substitute for application-level TCP measurements on the target systems.
- **Community reports:** Public IP-over-Thunderbolt results motivate the assumed range but vary by controller, kernel, topology, and test method; they are not treated as measurements of this deployment.

Table 9: GLM-5.3-Flash checkpoint size reconciliation.

Step	Size	Specification / Source
GLM-5.3-Flash BF16 (full model)	642 GB	320B total parameters $\times$ 2.01 B/w
GLM-5.3-Flash FP8 (full model)	321 GB	320B total parameters $\times$ 1.00 B/w
<i>Architecture and Sparsity:</i>		
Total parameters	$\sim$ 320B	45 layers (34 KDA + 11 DSA)
Routed experts	288	Top-8 activated per token ($K = 8$)
Active parameters per token	$\sim$ 18B	18.0B forward pass footprint
<i>Quantization Formats:</i>		
Q4_0_ROCMFP4_STRIX_LEAN	158.0 GB	$\sim$ 4.3 bpw ($\sim$ 79.0 GB per node)
UD-IQ3_XXS (single-node)	120.0 GB	Fits single 128 GB node
STRIX-BALANCED (compact)	72.0 GB	Fits compact 96 GB allocation
<i>Dual-Node Deployment Fit Check:</i>		
Cluster memory (2 $\times$ 128 GB)	256.0 GB	Hardware specification
Model checkpoint (ROCmFP4)	158.0 GB	Partitioned evenly (79 GB/node)
Hybrid DSA KV cache (200K tokens)	2.5 GB	1.27 GB per node (11 DSA layers)
Full MLA KV cache (200K tokens)	10.4 GB	5.18 GB per node (45 layers)
Remaining headroom (OS + buffers)	87.6–95.4 GB	Ample headroom for 30+ sessions

A.3 Checkpoint Size Reconciliation

The following derivation shows how the deployed GLM-5.3-Flash checkpoint size is obtained from public specifications and standard quantization bit-widths:

Key finding: Unlike dense 744B-class architectures that require destructive expert pruning to fit 256 GB, GLM-5.3-Flash fits natively under ROCmFP4 (Q4_0_ROCMFP4_STRIX_LEAN) at 158.0 GB gross without pruning any of its 288 experts, preserving 100% of model reasoning capacity while retaining over 87 GB of memory headroom across the dual-node cluster.

B Legal Disclaimer and Forward-Looking Statements

Nature of Document. This document is an academic preprint presenting an architectural feasibility study and formal mathematical analysis. It constitutes a *pre-competitive technical disclosure* intended for peer review, academic discussion, and preliminary investor due diligence. It is **not** a product specification, performance guarantee, binding technical commitment, or contractual representation of any kind.

Forward-Looking Statements. This paper contains forward-looking statements regarding projected performance, cost efficiency, throughput, and deployment feasibility of the Heterogeneous Compute Cascade (HCC) architecture. These projections are derived from roofline models, theoretical bounds, and published hardware specifications under idealized conditions. Actual results may differ materially due to, but not limited to: software implementation complexity, hardware manufacturing variability, thermal throttling under sustained loads, evolving vendor software support (ROCm, Vitis AI, ONNX Runtime), changes in third-party model availability (GLM-5.3-Flash, ROCmFP4, quantization toolchains), market pricing fluctuations, and regulatory or supply chain constraints. No representation is made that the projected performance will be achieved in any specific deployment.

No Warranty of Fitness. The author makes no warranty, express or implied, regarding the fitness of the HCC architecture for any particular commercial purpose. The economic comparisons (Tables 3–6) reflect publicly available pricing as of Q1 2026 and are subject to change. The experimental validation roadmap (Section 11) describes *planned* protocols; completion of these experiments is not guaranteed and is contingent on ongoing development.

Intellectual Property. The HCC architectural design, zero-copy DMA-BUF orchestration methodology, and multi-node speculative amortization strategy described herein constitute proprietary intellectual property of the author. Publication of this preprint does not constitute a grant of license, waiver of rights, or invitation to replicate the proprietary implementation. Open-source components referenced (ROCm, llama.cpp, TurboQuant, QuaRot) are governed by their respective licenses and are not claimed as proprietary.